ENGENHARIA DE SISTEMAS DISTRIBUÍDOS
DOCENTE: Raoni Kulesza
EQUIPE:
Bruno Formiga Brandão - 20210024863
Camila Eduardo Costa De Vasconcelos - 20220054768
Eliane Santos Silva - 20220155152
Gisele Silva Gomes - 20210025824
Klayvert De Andrade Araújo - 20220050259
Lucas Rondineli Lucena Fragoso - 20220005932

Documentação do Projeto

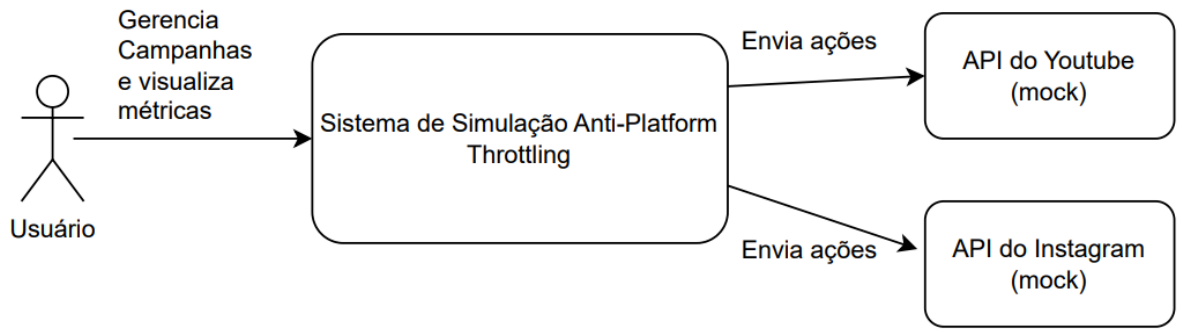
Repositório para o projeto:

https://github.com/eliane-silva/Projeto-ESD.git

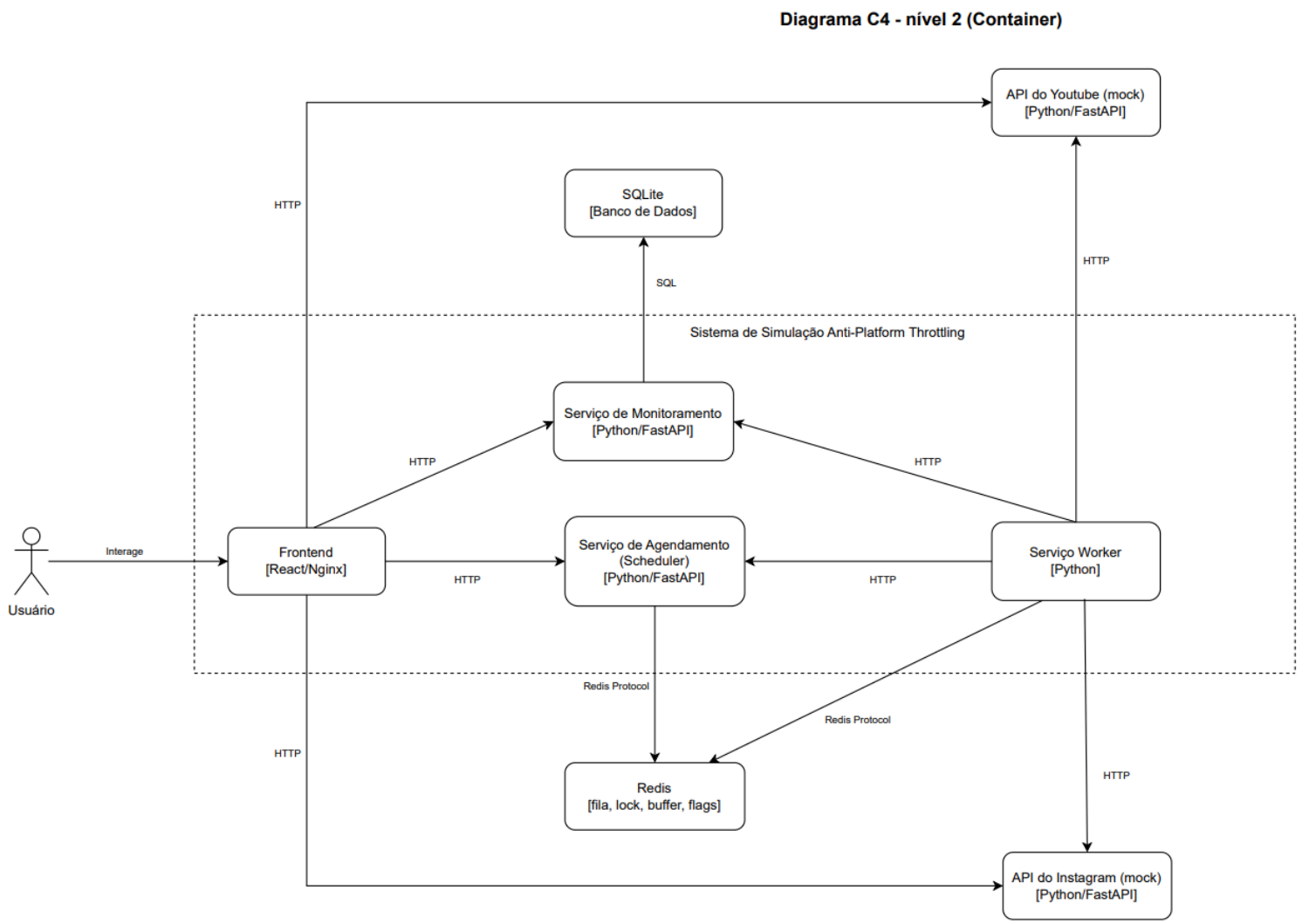
1. Diagrama C4

1.1 Diagrama C4 – Nível 1 (Contexto)

Diagrama C4 - nível 1 (Contexto)



1.2 Diagrama C4 – Nível 2 (Containers)



2. Decisões arquiteturais

2.1 ADR-001: Descoberta e aplicação de thresholds por plataforma

Cada plataforma simulada tem regras próprias de limitação de taxa. O sistema precisa descobrir, usar e atualizar um limite operacional seguro para evitar bloqueios frequentes e, ao mesmo tempo, maximizar throughput.

O sistema adota uma estratégia de ramp-up progressivo com teto dinâmico por plataforma. A lógica é:

1. iniciar com uma taxa baixa de ações por minuto;
2. aumentar gradualmente a taxa em janelas controladas;
3. ao detectar limitação ou bloqueio, registrar o evento;
4. reduzir o limite operacional para a plataforma;
5. aplicar cooldown com backoff exponencial;
6. retomar com taxa abaixo do último ponto de falha.

O organizador mantém:

- *safe_rate* estimado para cada plataforma;
- limite superior com erro confirmado para cada plataforma;

- histórico de falhas e sucessos.

Justificativa

- permite medir empiricamente os thresholds;
- adapta-se a diferentes plataformas;
- reduz necessidade de configuração manual fixa;
- combina segurança operacional com aprendizado experimental.

2.2 ADR-002: Estratégia de distribuição de ações entre plataformas

O sistema precisa distribuir um volume total de ações entre diferentes plataformas, respeitando limites individuais e evitando concentração excessiva em uma só plataforma.

Foi adotada uma estratégia de distribuição proporcional baseada na capacidade estimada de cada plataforma, com apoio de buffers para as campanhas. A distribuição segue os seguintes critérios:

- cada plataforma terá uma chance de ter uma campanha do buffer atendida proporcional ao seu *safe_rate*;
- campanhas ativas entram em uma fila de agendamento;
- o *scheduler* recalcula pesos ao fim de cada campanha;

Justificativa

- evita sobrecarga concentrada;
- adapta o fluxo ao comportamento observado;
- melhora utilização global da capacidade disponível;
- facilita a comparação entre plataformas.

2.3 ADR-003: Pausa automática diante de bloqueio

Quando uma plataforma começa a bloquear requisições, continuar insistindo aumenta a instabilidade do experimento e distorce as métricas.

Ao identificar padrão de bloqueio, o sistema ativará pausa automática por plataforma, combinada com reentrada gradual. Seguindo essas regras iniciais:

- primeiro bloqueio: pausar por 1 unidade de tempo;
- reincidência: 2, 4, 8, 16... até um teto configurável;
- após liberação, retornar com taxa reduzida;
- registrar evento para análise.

Justificativa

- protege a confiabilidade do experimento;
- evita avalanche de falhas;
- melhora a qualidade dos dados coletados;
- aproxima o sistema de um comportamento resiliente.

2.4 ADR-004: Acesso exclusivo às plataformas

Caso múltiplos workers fizessem requisições ao mesmo tempo nas plataformas, o limite de ações seria atingido mais rapidamente.

Por isso, o sistema faz com que em cada plataforma apenas uma campanha seja realizada por um worker por vez. Seguindo essas regras:

- um worker recebe uma campanha da fila;
- cada plataforma pode ter um worker trabalhando em uma campanha nela;
- campanhas novas adicionadas para plataformas que já tem uma campanha ativa são adicionadas temporariamente no buffer, não na fila geral;

Justificativa

- permite um cálculo mais coerente dos limites de cada plataforma;
- impede que o limite seja atingido rápido demais
- auxilia na distribuição da carga

3. Definição do stack tecnológico e justificativas

Linguagem: Python
Framework API: FastAPI
Fila e estado: Redis
Banco: SQLite
Containerização: Docker + Docker Compose
Frontend: React + ViteJS + Nginx

O projeto utiliza Python, FastAPI, Redis, SQLite, React, Nginx, Docker e Docker Compose, escolhidos por oferecerem simplicidade, integração entre serviços e facilidade de desenvolvimento. Python foi escolhido por sua sintaxe simples, alta produtividade e amplo ecossistema de bibliotecas. FastAPI foi adotado por facilitar a criação de APIs rápidas, bem organizadas e com documentação automática. Redis é utilizado como fila de mensagens e armazenamento de estado compartilhado (locks, buffers e feature flags), por ser leve e eficiente na comunicação entre scheduler e workers. SQLite foi definido como banco de dados por ser simples de configurar e suficiente para armazenar métricas e resultados do projeto. React + ViteJS foi escolhido devido a facilidade de desenvolvimento para telas no front-end para ser possível disponibilizar um painel de controle com design amigável ao usuário. Docker foi escolhido para padronizar o ambiente de execução e facilitar o uso em diferentes máquinas. Já o Docker Compose permite subir todos os serviços do sistema de forma integrada e centralizada.

4. Padrões arquiteturais implementados

4.1 Feature Flag

Área: Segurança / Deployment

O padrão Feature Flag está sendo utilizado para habilitar ou desabilitar funcionalidades do sistema de forma controlada, sem necessidade de alterar a arquitetura principal nem remover código já implementado. No nosso projeto, esse padrão é importante porque o ambiente é experimental e várias estratégias de

controle de engajamento podem ser ativadas gradualmente, comparadas entre si e desativadas rapidamente caso provoquem comportamento inadequado.

No nosso projeto, as feature flags são aplicadas principalmente sobre funcionalidades sensíveis do *scheduler* e do *worker*, como:

- modo de descoberta automática de thresholds por plataforma;
- ativação da pausa automática após bloqueios;
- uso de jitter na distribuição temporal das ações;
- redistribuição dinâmica de carga entre plataformas simuladas.

4.2 Rate Limit / Throttling Pattern

Área: Desempenho

O padrão Rate Limit / Throttling é um dos elementos centrais da arquitetura, pois o objetivo do projeto é justamente simular e medir estratégias para evitar excesso de interações em plataformas externas. Esse padrão consiste em controlar explicitamente a taxa de envio de requisições ou ações para que o sistema não ultrapasse limites aceitáveis de processamento ou detecção.

No nosso projeto, o rate limiting é aplicado em dois níveis. Nas plataformas mock, YouTube e Instagram impõem seus próprios limites de taxa, respondendo com HTTP 429 quando o limite é excedido. No lado do sistema, o Scheduler define o rate limit que cada campanha deve seguir, e o Worker aplica esse limite controlando o intervalo de tempo entre cada like enviado. Esse limite não é fixo, o Scheduler ajusta o valor entre campanhas usando uma estratégia de busca binária. Se o Worker completa uma campanha sem bloqueios, o Scheduler aumenta o rate limit para a próxima. Se houve bloqueios, diminui. Assim, o sistema se adapta ao comportamento observado de cada plataforma, convergindo progressivamente para o limite real.

4.3 Queues

Área: Escalabilidade

O padrão Queues está sendo utilizado para desacoplar a geração de ações da execução efetiva dessas ações nas plataformas simuladas. Em vez de o sistema enviar todas as interações diretamente no momento em que uma campanha é criada, as ações serão primeiro publicadas em filas, para depois serem consumidas pelos workers responsáveis por cada plataforma.

No nosso projeto, o Scheduler recebe a campanha via HTTP e a coloca na fila principal do Redis. Os Workers ficam escutando essa fila com BRPOP, um comando bloqueante que aguarda até uma tarefa aparecer. Quando a plataforma está ocupada, campanhas adicionais são armazenadas em buffers separados por plataforma e promovidas para a fila principal quando o lock é liberado. Qualquer Worker pode consumir qualquer campanha, independente da plataforma. Essa

arquitetura permite escalar o número de Workers de forma independente, aumentando o throughput do sistema sem alterar o Scheduler.

4.4 Circuit Breaker

Área: Confiabilidade

O padrão Circuit Breaker é utilizado para proteger o sistema contra falhas em cascata quando uma plataforma começa a bloquear requisições. Continuar enviando ações após um bloqueio aumenta a instabilidade e distorce as métricas do experimento.

No nosso projeto, o circuit breaker é implementado no worker. Ao receber uma resposta HTTP 429 (Too Many Requests), o worker ativa o circuito aberto e pausa com backoff exponencial:

- Primeiro bloqueio: pausa de 1 segundo;
- Reincidência: 2, 4, 8, 16 segundos, dobrando a cada bloqueio;
- O backoff é limitado por um teto configurável armazenado no Redis (config:max_pause_time, padrão 64 segundos);
- Após uma requisição bem-sucedida, o tempo de pausa reseta para 1 segundo.

Além disso, no primeiro bloqueio de cada campanha, o rate limit é reduzido pela metade. Cada bloqueio é registrado no serviço de monitoramento como evento e como ação com status "blocked", permitindo análise posterior. O circuit breaker é controlado pela feature flag circuit_breaker, que quando desligado, o worker não pausa após bloqueios, permitindo comparar o comportamento do sistema com e sem essa proteção.

5. Setup do ambiente

Estrutura final do ambiente do projeto:

```
project/
|-- docker-compose.yml
|-- main.py
|-- .env
|-- .env.docker
|-- services/
|   |-- frontend/
|   |   |-- components/
|   |   +-- pages/
|   |-- fake_youtube/
|   |   |-- app.py
```

```
|   |   +-- Dockerfile
|   |-- fake_instagram/
|   |   |-- app.py
|   |   +-- Dockerfile
|   |-- monitoring/
|   |   |-- app.py
|   |   |-- database.py
|   |   |-- models.py
|   |   +-- Dockerfile
|   |-- worker/
|   |   |-- worker.py
|   |   +-- Dockerfile
|   |-- scheduler/
|   |   |-- scheduler.py
|   |   +-- Dockerfile
|-- tests/
|   |-- chaos_test.py
|   |-- load_test.py
|   |-- test_scheduler.py
|   +-- test_worker.py
```

Conforme representado no diagrama C4, optou-se pela criação de duas plataformas simuladas (YouTube e Instagram), com o objetivo de permitir a execução de interações automatizadas em um ambiente controlado, evitando violações de termos de uso de plataformas reais.

A arquitetura é composta por um operador (frontend), que funciona como interface web (painel de controle) para o usuário gerenciar campanhas, flags e visualizar métricas e estatísticas; um *scheduler*, responsável pelo agendamento e distribuição das tarefas ao longo do tempo (incluindo estratégias como *jitter* e distribuição dinâmica); múltiplos *workers*, responsáveis por consumir a fila de tarefas no Redis e executar as interações nas APIs simuladas; e um serviço de monitoramento, responsável por registrar cada ação e evento em um banco de dados SQLite.

O Redis é utilizado como infraestrutura externa para armazenamento da fila de campanhas, *locks* de plataforma, *buffers* e *feature flags*. O serviço de monitoramento persiste informações como taxa de sucesso, erros e ocorrências de throttling no SQLite, permitindo a análise do comportamento do sistema e validação das estratégias de controle de carga implementadas.

As URLs dos serviços são centralizadas em arquivos de variáveis de ambiente (.env para execução local e .env.docker para execução dentro dos containers), evitando URLs hardcoded no código.

6. Divisão de responsabilidades entre os integrantes

Inicialmente, a divisão de responsabilidades foi organizada de modo que cada integrante ficasse responsável por um serviço ou módulo principal do sistema. Um integrante ficou encarregado do Scheduler, incluindo a lógica de agendamento e distribuição de tarefas, outro das APIs simuladas do YouTube e Instagram, outro da interface do operador, outro do monitoramento e persistência em banco de dados, outro dos Workers responsáveis pela execução assíncrona, e o último dos testes, integração final e validação da solução.

No entanto, cada integrante acabou auxiliando em outras áreas além da que ficou encarregado:

Gisele Silva Gomes:

- Operador + Redis + Diagramas + Documentação
- circuit_breaker, melhorias no operador e scheduler, validação de entradas de dados, merge e resolução de conflitos

Eliane Santos Silva

- YouTube Mock + Instagram Mock + Diagramas + Documentação
- Validações de entrada de dados e tratamento de erros
- Alterações no scheduler e worker para suportar a mudança na quantidade de vídeos que antes só tínhamos um por plataforma.

Bruno Formiga Brandão

- Integração do backend com o Frontend
- Front end: Criação das telas de Dashboard, Campanhas, Feature Flags, Estatísticas dos vídeos

Camila Eduardo Costa de Vasconcelos

- Monitoramento + banco

Lucas Rondineli Lucena Fragoso

- Auxílio na criação das ADRs, padrões arquiteturais, stack e setup;
- Implementação inicial do menu do operador, da lógica da estimativa do *scheduler*, dos buffers de cada plataforma, das flags utilizadas e jitter nos workers;

Klayvert De Andrade Araújo

- Implementação do ambiente Docker, Docker Compose, conexões iniciais com as plataformas e testes.

7. Trade-offs documentados com justificativas

7.1 Plataformas simuladas versus integração com plataformas reais

Decisão: Usar APIs mock de YouTube e Instagram em vez de integrar com serviços reais

Foi decidido utilizar plataformas simuladas, representadas pelos serviços Mock YouTube e Mock Instagram, em vez de integração com APIs reais de plataformas

externas. Essa escolha foi adotada para permitir experimentação em ambiente controlado, sem dependência de autenticação externa, sem risco de violação de termos de uso e sem interferência de fatores imprevisíveis do mundo real. Além disso, essa decisão permite configurar regras de throttling específicas para cada plataforma simulada, o que favorece a análise do comportamento do sistema diante de diferentes limites operacionais. Como desvantagem, o ambiente se torna menos fiel ao comportamento real de plataformas externas, já que os bloqueios e thresholds são definidos por regras simplificadas. Ainda assim, para os objetivos do projeto, a utilização de simuladores oferece maior segurança, reprodutibilidade e controle experimental.

7.2 Redis como fila versus banco relacional ou broker mais robusto

Decisão: Utilizar o Redis como mecanismo principal de fila entre scheduler e workers

Foi decidido utilizar o Redis pois ele oferece baixa latência, simplicidade e facilidade de uso no fluxo do projeto. Isso combina bem com a necessidade de enfileirar campanhas, manter buffers por plataforma e permitir consumo pelos workers de forma rápida. No nosso projeto, o Redis é usado tanto para a fila principal quanto para locks, buffers e feature flags, o que reduz a quantidade de componentes diferentes na solução. Como desvantagens, na nossa aplicação, ele não oferece as mesmas garantias de durabilidade, rastreabilidade e semântica avançada de mensageria que seriam esperadas em soluções como, por exemplo, Kafka. A equipe priorizou simplicidade de desenvolvimento, facilidade de execução e baixo custo operacional em vez de resiliência máxima da camada de mensageria. Para essa POC, pensamos que isso reduz a complexidade sem descaracterizar os conceitos distribuídos centrais do projeto.

7.3 SQLite para persistência versus PostgreSQL ou outro SGBD servidor

Decisão: Utilizar SQLite para armazenar ações, eventos e métricas do sistema

Decidimos usar o SQLite pois ele simplifica a configuração do ambiente, elimina a necessidade de administrar um servidor de banco separado e atende bem ao uso de persistência leve para métricas e eventos de monitoramento. No nosso projeto, ele armazena registros de ações executadas e eventos como bloqueios, aumento e redução de taxa, o que faz uma excelente análise experimental do comportamento do nosso sistema. Como desvantagens, SQLite possui limitações de concorrência e escalabilidade de escrita quando comparamos a um banco cliente-servidor como por exemplo o PostgreSQL. Com muitos workers, múltiplas gravações simultâneas e necessidade de consultas mais pesadas, esse banco pode se tornar um gargalo. A nossa decisão foi pela facilidade de setup e velocidade de entrega em detrimento de escalabilidade.

7.4 Controle adaptativo de rate limit versus limite fixo por plataforma

Decisão: O scheduler trabalha com ajuste progressivo de taxa, aumentando ou reduzindo o rate limit de acordo com o resultado das campanhas, em vez de usar um limite fixo estático

Foi decidido esse modelo pois quando uma campanha é aprovada, o nosso sistema tenta aumentar a taxa e, quando há rejeição ou bloqueio, ele reduz. Isso faz sentido com a proposta do nosso projeto de descobrir thresholds de forma dinâmica e adaptar o comportamento conforme a resposta observada do ambiente. Como desvantagem, a estratégia é mais complexa do que um limite fixo. Ela exige manutenção de estado, eventos de monitoramento e mecanismos de ajuste que podem oscilar ou convergir lentamente, especialmente em ambientes com um comportamento muito variável. A equipe optou por maior complexidade de controle para ganhar adaptabilidade e valor experimental, já que o objeto do projeto não é apenas a execução de ações, mas também medir e compreender o limiar de throttling das plataformas simuladas.

7.5 Circuit breaker com pausa versus retentativas agressivas

Decisão: Quando o worker recebe resposta 429, o sistema pode abrir o circuito e aplicar pausas progressivas até um teto configurável

Decidimos isso pois esse padrão reduz a cascata de falhas, evita insistência imediata após bloqueio e preserva a qualidade dos dados observados. No nosso projeto, o worker detecta bloqueio, registra evento, reduz a taxa e, se a flag estiver ativa, abre o circuito com crescimento do tempo de pausa. O teto dessa pausa também pode ser ajustado pelo operador. A principal desvantagem é a perda de throughput no curto prazo. Além disso, uma pausa excessiva pode fazer o sistema reagir de forma conservadora demais, alongando o tempo necessário para concluir campanhas. A solução decidida pela equipe privilegia resiliência e proteção contra instabilidade em vez de agressividade na execução.

7.6 Lock e buffer por plataforma versus paralelismo irrestrito

Decisão: Usar lock por plataforma e buffers separados, evitando que múltiplas campanhas para a mesma plataforma sejam processadas ao mesmo tempo sem coordenação

Foi decidido por essa abordagem pois implementa isolamento por plataforma. Se o Youtube estiver sob bloqueio, por exemplo, isso não impede o Instagram de continuar processando campanhas. O lock reduz a disputa descontrolada sobre o mesmo alvo e facilita redistribuição. O custo direto é a redução de paralelismo por

plataforma. Em vez de explorar a máxima concorrência, o sistema favorece a previsibilidade e controle. Isso pode diminuir o throughput. A equipe priorizou estabilidade e isolamento de falhas em vez de vazão máxima.

7.7 Jitter temporal versus envio em intervalos fixos

Decisão: Permitir jitter no intervalo entre ações, em vez de manter espaçamento estritamente constante

Decidimos pelo jitter pois ele evita padrões rígidos e artificiais de envio, deixando o comportamento do sistema mais próximo de um fluxo menos previsível. Na implementação, quando a flag está ativa, o worker sorteia o tempo de espera em torno do atraso médio calculado a partir do rate limit. Ao introduzir variabilidade, o sistema perde parte da previsibilidade temporal exata. Isso pode dificultar testes totalmente determinísticos e comparações muito rígidas entre execuções. A equipe preferiu realismo comportamental e redução de padrões detectáveis em vez de temporização perfeitamente determinística.

7.8 Feature flags versus comportamento rígido no código

Decisão: Ativar ou desativar dinamicamente funcionalidades como threshold discovery, distribuição dinâmica, jitter e circuit breaker por meio de flags armazenadas no Redis

Foi decidido isso pois essa decisão aumenta muito a capacidade experimental do projeto. O operador consegue comparar estratégias diferentes sem reescrever a arquitetura ou alterar manualmente o fluxo principal. Feature flags aumentam a quantidade de caminhos de execução e, portanto, a complexidade de teste e manutenção. A equipe optou por flexibilidade experimental e facilidade de validação comparativa em vez de um comportamento único e mais simples.

7.9 Docker Compose para orquestração local versus Kubernetes ou infraestrutura mais sofisticada

Decisão: Subir todo o ambiente com Docker Compose

Decidimos por Docker Compose pois ele simplifica a execução integrada dos serviços, padroniza o ambiente e facilita a reprodução do experimento por qualquer integrante da equipe. Isso é visível no projeto, que sobe Redis, scheduler, worker, monitoramento e as duas plataformas fake em conjunto. A solução perde recursos mais avançados de orquestração, auto-healing, escalonamento horizontal inteligente, políticas de rede e observabilidade nativa que seriam comuns em plataformas como Kubernetes. A equipe priorizou portabilidade local e simplicidade operacional em vez de uma infraestrutura de produção.

8. Documentação de Testes e Validação

8.1 Validação de Regras de Negócio e Lógica de Controle

A primeira camada de testes da POC 5 teve como objetivo homologar o núcleo matemático e as regras de roteamento da arquitetura de Anti-Platform Throttling. Para garantir que a lógica estivesse livre de interferências de rede ou latência de banco de dados, foi desenvolvida uma suíte de testes isolados utilizando o framework **pytest** e a biblioteca **unittest.mock**.

Através da técnica de *Mocking*, simulou-se o comportamento do Message Broker (Redis) e dos serviços externos alvo (via **httpx**). A suíte de validação foi dividida em dois domínios principais:

Testes do Domínio de Recepção (Scheduler):

- **Controle de Concorrência (test_lock_e_buffer_de_campanhas):** Comprovou a eficácia do mecanismo de exclusão mútua. O teste validou que o algoritmo adquire um *Lock* no Redis para a primeira campanha de uma plataforma e direciona corretamente campanhas simultâneas para a mesma plataforma rumo ao *Buffer* (Fila de Espera), evitando o processamento sobreposto.
- **Algoritmo de Vazão (test_throttle_diminui_rate_limit):** Validou o *feedback loop*. Ao simular o recebimento de uma flag de falha do worker, o teste garantiu que o Scheduler calculasse e reduzisse a velocidade máxima de disparos (rate_limit) adequadamente.

Testes do Domínio de Processamento (Worker):

- **Parsing e Roteamento (test_worker_parsing_logic e test_worker_selection_logic):** Garantiram que o worker extrai corretamente os metadados da fila do Redis e constrói dinamicamente as URLs de destino corretas (ex: separando tráfego do YouTube e do Instagram).
- **Cálculo de Pausa Exponencial (test_worker_circuit_breaker_logic_mocked):** Homologou a resposta a falhas. Simulando uma situação de bloqueio (Erro 429), atestou-se que o *Circuit Breaker* calcula a pausa exponencial multiplicando o tempo de espera corretamente, sem ultrapassar o limite máximo configurado (max_pause_time).

- **Imagem:** Print do terminal exibindo a execução do Pytest com os 5 testes retornando a flag verde PASSED, comprovando que a lógica base está íntegra de forma isolada.

```
tests/test_scheduler.py::test_lock_e_buffer_de_campanhas PASSED          [ 20%]
tests/test_scheduler.py::test_throttle_diminui_rate_limit PASSED         [ 40%]
tests/test_worker.py::test_worker_parsing_logic PASSED                   [ 60%]
tests/test_worker.py::test_worker_selection_logic PASSED                 [ 80%]
tests/test_worker.py::test_worker_circuit_breaker_logic_mocked PASSED    [100%]
```

8.2 Simulação de Tráfego e Comportamento de Usuário (Carga e Stress)

Para a execução dos testes de carga e validação do Anti-Platform Throttling, foi desenvolvido um script de simulação de tráfego utilizando o framework Locust (load_test.py). O objetivo deste script é modelar o comportamento estocástico (aleatório e imprevisível) de múltiplos clientes interagindo concorrentemente com o Scheduler.

Modelagem do Usuário Simulador (CampaignUser):

Concorrência e Cadência: O tempo de espera entre as requisições de um mesmo usuário foi configurado entre 1 e 2 segundos (wait_time = between(1, 2)), simulando o tempo de navegação humana e evitando sobrecarga artificial na porta de entrada da aplicação.

Distribuição de Carga (Pesos): As tarefas foram distribuídas utilizando o decorador @task com pesos específicos para simular um cenário de uso real:

- **Peso 3:** Criação de campanhas simultâneas para o YouTube (/start_campaign?platform=youtube).
- **Peso 3:** Criação de campanhas simultâneas para o Instagram (/start_campaign?platform=instagram).
- **Peso 1:** Requisições de Health Check (/docs), simulando usuários consultando o status do painel.
- **Aleatoriedade de Carga (Payload):** Para evitar o cacheamento de rotas e forçar o processamento dinâmico, os parâmetros de query (como **content_id** e a quantidade de **actions**) são gerados aleatoriamente a cada requisição HTTP através da biblioteca **random.**

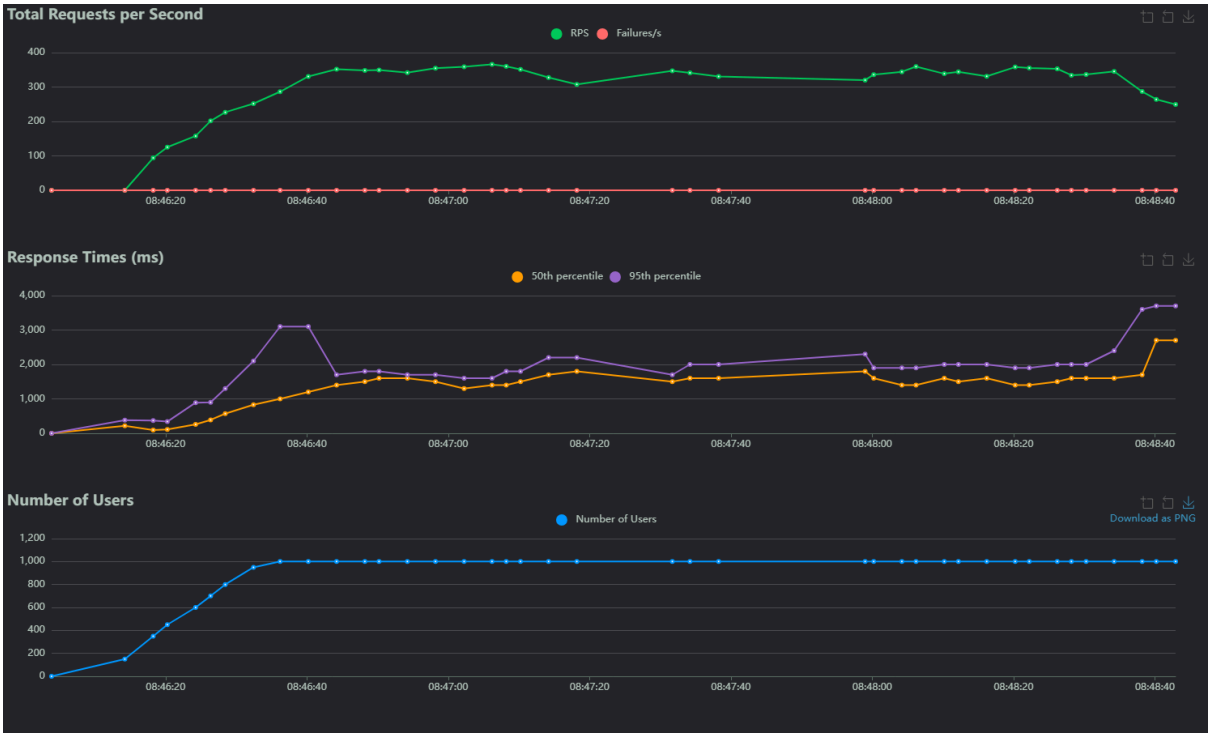
A interface do Locust foi configurada para criar uma rampa de acesso de **50 novos usuários por segundo**, até atingir o pico de **1.000 usuários simultâneos**. Considerando o tempo de espera simulado no código (wait_time de 1 a 2 segundos) somado ao tempo de resposta do servidor sob estresse (aproximadamente 1.5s a 2s), esse contingente gerou uma vazão média estabilizada na casa das 250 RPS (Requisições Por Segundo). Este volume de tráfego foi projetado especificamente para saturar rapidamente a fila do Redis, obrigando os Workers escalados a

atingirem o limite rígido das plataformas destino e acionarem a lógica de degradação graciosa.

Imagem: Print da tabela "Statistics" do Locust, evidenciando o processamento de alto volume com taxa de falhas (# Fails) igual a 0, atestando a absorção de carga pelo Scheduler.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/start_campaign [instagram]	20349	0	1600	2300	3600	1585.18	4	4713	108	105.5	0
POST	/start_campaign [youtube]	20482	0	1600	2300	3600	1583.52	4	4879	104	102.5	0
GET	Health Check (Docs)	6920	0	140	430	1200	190.52	3	3706	1017	41.5	0
	Aggregated	47751	0	1500	2200	3600	1382.36	3	4879	238.02	249.5	0

Imagem: Print da aba "Charts" do Locust, demonstrando a rampa de crescimento de usuários (estabilizando em 1000) e a manutenção da vazão (RPS) constante sem indisponibilidade.



8.3 Metodologia de Injeção de Falhas Automatizada (Chaos Scripting)

Para garantir a repetibilidade e a precisão da validação de resiliência da POC 5, foi desenvolvido um script de automação focado em Engenharia do Caos (test_chaos.py). A ferramenta foi construída em Python, utilizando a biblioteca subprocess para realizar chamadas diretas ao daemon do Docker, contornando a aplicação e atacando a infraestrutura subjacente de forma programática.

A automação orquestra a injeção de duas falhas críticas em sequência, enquanto o sistema está sob carga induzida (via Locust):

Simulação de Queda do Message Broker (test_chaos_redis_restart): O script emite o comando de parada abrupta para o contêiner do banco de dados (docker compose stop redis). O objetivo é induzir uma falha de conexão generalizada (ConnectionRefusedError) em todos os Workers e no Scheduler. A automação impõe um intervalo de 5 segundos de *downtime* antes de reiniciar o serviço, tempo exato para homologar se o mecanismo de *self-healing* (auto-recuperação) dos Workers mantém os processos em loop de espera ao invés de encerrar a aplicação com erro fatal.

Descrição da Imagem: Print do log do sistema no momento da queda do Redis. Demonstra o Worker interceptando a falha de rede ("Tentando reconectar...") enquanto o Scheduler falha rapidamente (Erro 500), evidenciando o isolamento adequado de responsabilidades assíncronas e síncronas.

```
redis-1           | 1:C 02 Apr 2026 11:09:41.180 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
worker-4          | [youtube/youtube_video_5] Ação 6 OK
redis-1           | 1:C 02 Apr 2026 11:09:41.180 * Redis version=7.4.8, bits=64, commit=00000000, modified=0, pid=1, just started
redis-1           | 1:C 02 Apr 2026 11:09:41.180 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.co
redis-1           | 1:M 02 Apr 2026 11:09:41.181 * monotonic clock: POSIX clock_gettime
fake_instagram-1  | INFO:     172.19.0.6:44054 - "POST /api/instagram/like?video_id=instagram_video_2 HTTP/1.1" 200 OK
fake_youtube-1    | INFO:     Started server process [1]
fake_youtube-1    | INFO:     Waiting for application startup.
fake_youtube-1    | INFO:     Application startup complete.
worker-4          | [youtube/youtube_video_5] Ação 7 OK
worker-4          | [youtube/youtube_video_5] Ação 8 OK
fake_instagram-1  | INFO:     172.19.0.6:44054 - "POST /api/instagram/like?video_id=instagram_video_2 HTTP/1.1" 200 OK
scheduler-1       | INFO:     172.19.0.1:36388 - "POST /start_campaign?platform=instagram&actions=91&content_id=instagram_video_1 HTTP/1.1" 200 OK
worker-1          | [youtube/youtube_video_1] Ação 2 OK
scheduler-1       | INFO:     172.19.0.1:36358 - "POST /start_campaign?platform=instagram&actions=64&content_id=instagram_video_2 HTTP/1.1" 200 OK
worker-1          | [youtube/youtube_video_1] Ação 3 OK
worker-1          | [youtube/youtube_video_1] Ação 4 OK
scheduler-1       | INFO:     172.19.0.1:36402 - "POST /start_campaign?platform=youtube&actions=34&content_id=youtube_video_3 HTTP/1.1" 200 OK
scheduler-1       | INFO:     172.19.0.1:36374 - "POST /start_campaign?platform=instagram&actions=24&content_id=instagram_video_2 HTTP/1.1" 200 OK
worker-4          | [youtube/youtube_video_5] Ação 9 OK
worker-4          | [youtube/youtube_video_5] Ação 10 OK
worker-4          | [youtube/youtube_video_5] Ação 11 OK
worker-4          | [youtube/youtube_video_5] Ação 12 OK
worker-4          |
worker-4          | [WORKER] Conexão com o Redis perdida! Tentando reconectar em 5s...
```

Simulação de Falha de Hardware / Nó Morto (test_chaos_kill_worker): Para simular a queima de um servidor físico ou uma falha de kernel irrecuperável, o script rastreia os Process IDs (PIDs) ativos na rede do Docker e executa um **docker kill** contra uma instância aleatória da camada de processamento. Ao contrário de um encerramento gracioso (*graceful shutdown*), o comando envia um sinal **SIGKILL**, eliminando o Worker instantaneamente. Esta etapa valida arquiteturalmente a premissa de descentralização: a queda de um nó não deve corromper a fila e os nós sobreviventes devem continuar consumindo as mensagens do Redis sem interrupção do serviço global.

```
--- TESTE 2: Matando um Worker aleatório ---
Executando: docker ps --format {{.ID}} {{.Names}}
Matando worker 9b139dfbb3cc...
Executando: docker kill 9b139dfbb3cc
Worker morto. Verificando se a fila continua sendo processada por outros...
```

```
=== TESTES DE CAOS FINALIZADOS ===
PS C:\ESD\Projeto-ESD> docker compose ps
NAME                      IMAGE                     COMMAND                  SERVICE          CREATED          STATUS          PORTS
projeto-esd-mock_instagram-1   projeto-esd-mock_instagram   "python -u -m uvicor…"   mock_instagram   2 minutes ago    Up 2 minutes    0.0.0.0:8002->8000/tcp, [::]:8002->80
00/tcp
projeto-esd-mock_youtube-1     projeto-esd-mock_youtube     "python -u -m uvicor…"   mock_instagram   2 minutes ago    Up 2 minutes    0.0.0.0:8002->8000/tcp, [::]:8002->80
00/tcp
projeto-esd-mock_youtube-1     projeto-esd-mock_youtube     "python -u -m uvicor…"   mock_youtube     2 minutes ago    Up 2 minutes    0.0.0.0:8001->8000/tcp, [::]:8001->80
00/tcp
projeto-esd-monitoring-1       projeto-esd-monitoring       "uvicorn app:app --h…"   monitoring       2 minutes ago    Up 2 minutes    0.0.0.0:8003->8000/tcp, [::]:8003->80
00/tcp
projeto-esd-redis-1            redis:7-alpine               "docker-entrypoint.s…"   redis            31 minutes ago   Up 38 seconds   0.0.0.0:6379->6379/tcp, [::]:6379->63
79/tcp
projeto-esd-scheduler-1        projeto-esd-scheduler        "python -u -m uvicor…"   scheduler        2 minutes ago    Up 2 minutes    0.0.0.0:8000->8000/tcp, [::]:8000->80
00/tcp
n -u -m uvicor…"   mock_youtube     5 minutes ago    Up 5 minutes                0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp
projeto-esd-monitoring-1       projeto-esd-monitoring       "uvicorn app:app --h…"   monitoring       5 minutes ago    Up 5 minutes                0.0.0.0:8003->8000/tcp,
[::]:8003->8000/tcp
projeto-esd-redis-1            redis:7-alpine               "docker-entrypoint.s…"   redis            34 minutes ago   Up 3 minutes                0.0.0.0:6379->6379/tcp,
[::]:6379->6379/tcp
projeto-esd-scheduler-1        projeto-esd-scheduler        "python -u -m uvicor…"   scheduler        5 minutes ago    Up 5 minutes                0.0.0.0:8000->8000/tcp,
[::]:8000->8000/tcp
projeto-esd-worker-1           projeto-esd-worker           "python -u worker.py"    worker           5 minutes ago    Up 5 minutes
projeto-esd-worker-2           projeto-esd-worker           "python -u worker.py"    worker           5 minutes ago    Up 5 minutes
projeto-esd-worker-3           projeto-esd-worker           "python -u worker.py"    worker           5 minutes ago    Exited (137) 3 minutes ago
```

Descrição da Imagem: Print do terminal com o resultado do comando docker compose ps -a após a execução do script de caos. A imagem apresenta a tabela de estado dos contêineres, onde se observa o projeto-esd-worker-3 com o status crítico Exited (137), confirmando sua interrupção abrupta via SIGKILL induzida pelo teste. Simultaneamente, a imagem comprova a resiliência da arquitetura ao mostrar que os nós worker-1 e worker-2, assim como o scheduler-1 e o redis-1, permanecem com o status Up, garantindo a continuidade do processamento da fila sem interrupção do serviço global.